

CS 423

Operating System Design: Concurrency: Locks

Oct 06

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

ANNOUNCEMENTS

MP2 Due 10/20 11:59 pm CT

MP1 grades will be released later today

Thursday lecture 10/08 – via recording (conference travel) -
no need to come to lecture, watch asynchronously
Will post link to recording on Piazza Thursday morning

AGENDA / LEARNING OUTCOMES

Concurrency:

What is the motivation for concurrent execution

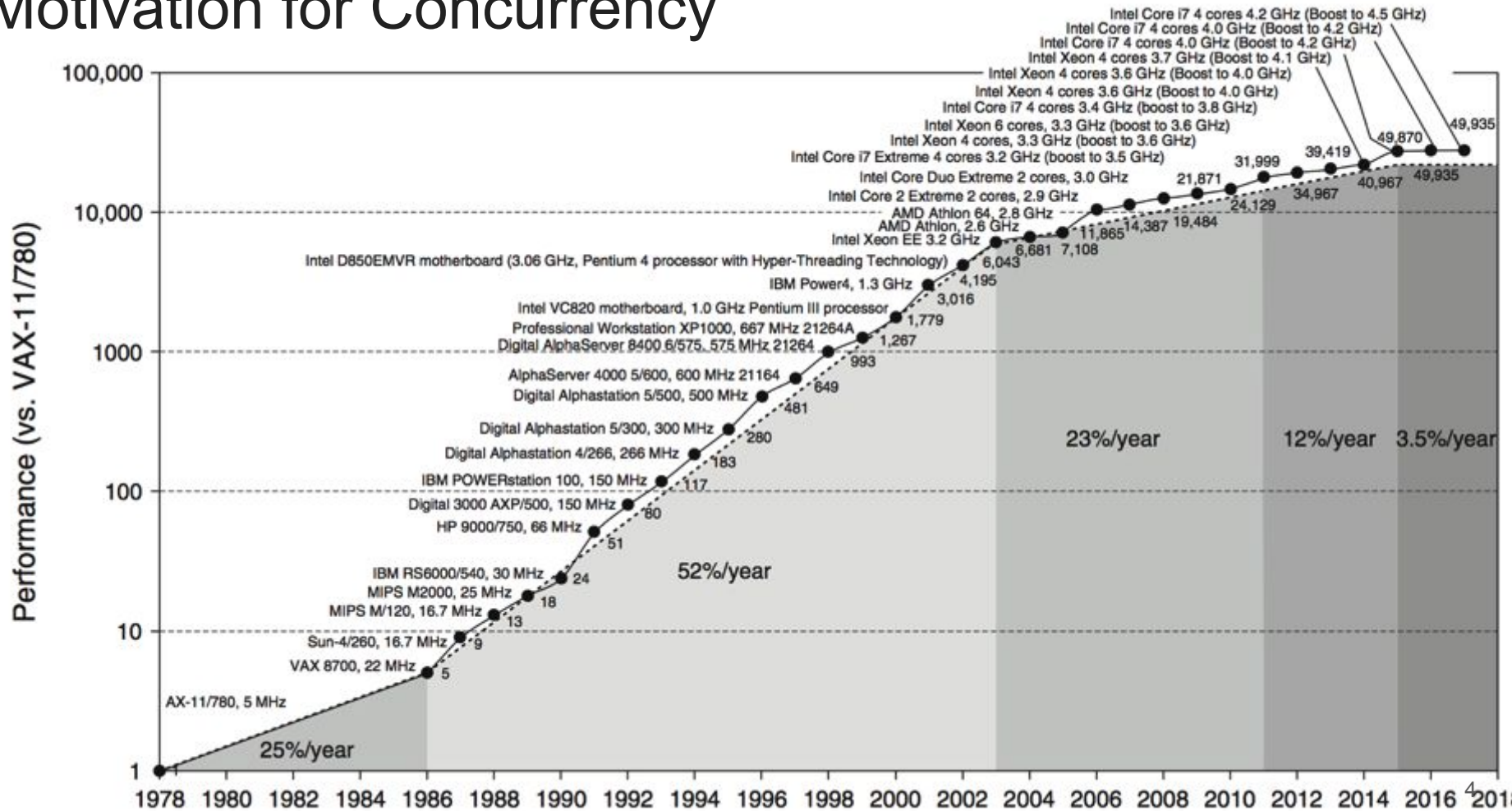
Challenges?

Threading & Locks

For people that have taken 341 (or its equivalent)

Some of this will be obvious

Motivation for Concurrency



MOTIVATION

CPU Trend: Speed has stabilized; increasing #cores

Goal: Applications utilize more cores

Option 1: Build app with many **processes**

- Example: Browser (process per tab)

Pros:

- Don't need new abstractions
- Good for security

Cons:

- High communication overhead
- Expensive context switching

Option 2: Build app with many **threads**

- Example: Word processors, video games

Pros:

- Shared address space
- Cheaper: fast context switch
- Easier communication

Cons:

- Data corruption if not careful

Many modern apps are hybrid: use both options
Scale up vs. scale out?

THREADING

New abstraction: thread

Threads are like “lightweight processes”, except:

Multiple threads (of same process) share the address space

Each thread has its own stack

Inter-thread communication is cheaper (than inter-process communication)

Don't need to send the actual data because the heap is shared

POSIX thread library, aka **pthread**s

THREADS WITHIN A PROCESS

All threads **share**:

- Process ID (PID)
- Page Table (PTBR), Address space: code (instructions), data (heap)
- Open file descriptors
- Current working directory
- User and group id

Each thread has its own **unique**:

- Thread ID (TID)
 - Set of registers, including Program Counter and Stack Pointer
 - Stack for local variables and return addresses (in same address space)
- Does the OS/hardware provide any protection against a thread accessing another's stack?

Threading: Approach 1

User-level threads: Many-to-one thread mapping

- Implemented by user-level runtime libraries
Create, schedule, synchronize threads at user-level
- OS is not aware of user-level threads
OS thinks each process contains only a single thread of control

Advantages

- Does not require OS support; Portable
- Can tune policies to suit application
- Lower overhead thread operations since no system call

Disadvantages?

- Cannot leverage multiprocessors
- Entire process blocks when one thread blocks

Threading: Approach 2

Kernel-level threads: One-to-one thread mapping

- User-level runtime lib, but each user-level thread gets a kernel thread
- Each kernel thread schedulable independently
- OS performs thread operations (create, schedule, synchronize)

Pros:

- Multiple threads can run in parallel
- When a thread blocks, other threads are unaffected

Cons:

- Higher overhead for thread operations (compared to Approach 1)

Common Programming Models

Typical programming models for threaded programs:

- **Producer/consumer**
Multiple producer threads create data (or work) that is handled by one of the multiple consumer threads
- **Pipeline**
Task is divided into series of subtasks, each of which is handled in series by a different thread
- **Deferred work done in the background**
One thread performs non-critical work in the background (when CPU idle)

THREAD SCHEDULE

```
volatile int balance = 0;
int loops = 1000000; // 1 million

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        balance++;
    }
    pthread_exit(NULL);
}
```

```
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("Initial value : %d\n", balance);
    pthread_create(&p1, NULL, worker, NULL);
    pthread_create(&p2, NULL, worker, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("Final value    : %d\n", balance);
    return 0;
}
```

A Schedule that leads to 2M

Bal++ boils down to:

```
mov eax, mem_addr(bal)
add 1, eax
mov mem_addr(bal), eax
```

Remember registers are virtualized...
Each thread has its own “copy” of eax
although there’s physically one eax on
the hardware

Initial balance = 0

Thread A:

repeat 1,000,000 times:

load balance

+1

store

balance = 1,000,000

Thread B:

repeat 1,000,000 times:

load balance

+1

store

balance = 2,000,000

A Schedule that leads to 1M

start: balance = 0

T1 loads 0

T2 loads 0

T1 stores 1

T2 stores 1

```
mov eax, mem_addr(bal)
add 1, eax
mov mem_addr(bal), eax
```

T1 loads 1

T2 loads 1

T1 stores 2

T2 stores 2

...

Can this happen even on a single core?

What's the lowest value possible?

Chat with your neighbors for two mins...(clue is here)

What's the schedule for this lowest possible value?

NON-DETERMINISM

Concurrency leads to non-deterministic results

- Different results even with same inputs
- Race conditions

Whether a bug manifests depends on the CPU schedule.

How to program: imagine scheduler is behaving maliciously.

General principle: Assume the worst-case schedule!

WHAT DO WE WANT?

Want 3 instructions to execute as an un-interruptable group

That is, we want them to be **atomic**

```
mov 0x123, %eax  
add $0x1, %eax  
mov %eax, 0x123
```

More generally: need mutual exclusion for critical sections

If thread A is in the critical section, all other threads cannot
(okay if other threads do unrelated work)

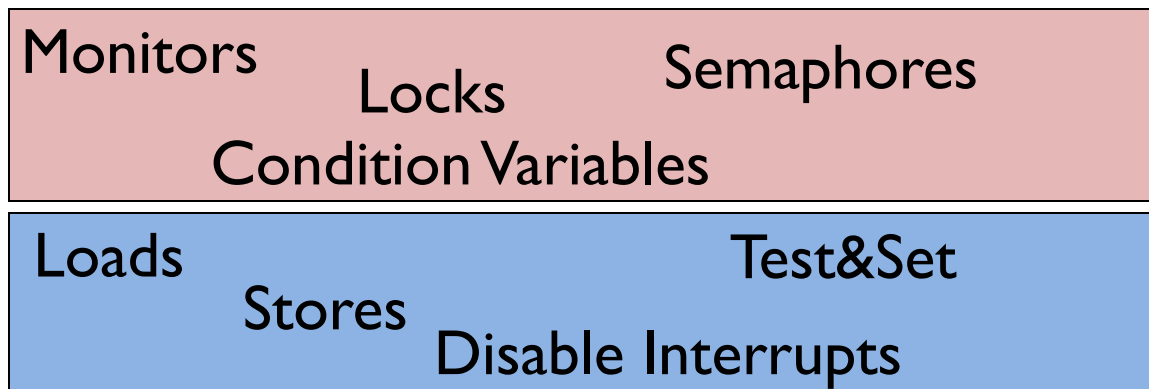
Synchronization

Build higher-level synchronization primitives in OS

Operations that ensure correct ordering of instructions across threads

Use help from hardware

Motivation: Build them once and get them right



LOCKS

Locks

Goal: Provide mutual exclusion (**mutex**)

Allocate and Initialize

- **Pthread_mutex_t** mylock = PTHREAD_MUTEX_INITIALIZER;

Acquire

- Acquire exclusion access to lock;
- Wait if lock is not available (some other process in critical section)
- Spin or block (relinquish CPU) while waiting
- **Pthread_mutex_lock**(&mylock);

Release

- Release exclusive access to lock; let another process enter critical section
- **Pthread_mutex_unlock**(&mylock);

LOCK IMPLEMENTATION GOALS

Correctness

- *Mutual exclusion (safety)*
Only one thread in critical section at a time
- *Progress (liveness)*
If several simultaneous requests, must allow one to proceed

Fairness: does each thread have a fair shot at acquiring? Does anybody starve?

Performance:

CPU is not used unnecessarily (spinning)

Only one thread, multiple threads, multiple cores

Implementing Locks

Approaches

- Disable interrupts
- Load and stores of words
- Using atomic hardware instructions (e.g., test and set)

Implementing Locks: W/ Interrupts

Turn off interrupts for critical sections

- Prevent dispatcher from running another thread
- Code between interrupts executes atomically

```
void acquire(lockT *l) {  
    disableInterrupts();  
}
```

```
void release(lockT *l) {  
    enableInterrupts();  
}
```

Disadvantages?

Implementing Locks: W/ Interrupts

Turn off interrupts for critical sections

- Prevent dispatcher from running another thread
- Code between interrupts executes atomically

```
void acquire(lockT *l) {  
    disableInterrupts();  
}
```

```
void release(lockT *l) {  
    enableInterrupts();  
}
```

Disadvantages?

Only works on uniprocessors

Process can keep control of CPU for arbitrary length

Turning off interrupts can lead to serious issues due to lost interrupts

Implementing LOCKS: w/ Load+Store

```
typedef struct __lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1) // TEST the flag
        ; // spin-wait (do nothing)
    mutex->flag = 1;         // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```

Does this provide mutual exclusion on multiple cores?

How about a single core?

What about performance? (esp in single core)

Using only Loads and Stores

Peterson's algorithm - uses only atomic load and store instructions to implement locks

Cumbersome to reason about correctness

But no lock is implemented this way today...

Most locks assume some support from hardware (or even OS)

XCHG: ATOMIC EXCHANGE OR TEST-AND-SET

How do we solve this ? Get help from the hardware!

```
// xchg(int *addr, int newval)
// return what was pointed to by addr
// at the same time, store newval into addr
int xchg(int *addr, int newval) {
    int old = *addr;
    *addr = newval;
    return old;
}
```

Test: return old value, **Set:** set addr with passed in value, HW does them atomically

LOCK Implementation with XCHG

```
typedef struct __lock_t {  
    int flag;  
} lock_t;
```

```
void init(lock_t *lock) {  
    lock->flag = 0;  
}
```

int **xchg**(int *addr, int newval)

```
void acquire(lock_t *lock) {  
    ???;  
    // spin-wait (do nothing)  
}
```

Chat with neighbors for 2 mins...

```
void release(lock_t *lock) {  
    lock->flag = ??;  
}
```

Other Atomic HW Instructions

```
int CompareAndSwap(int *addr, int expected, int new) {  
    int actual = *addr;  
    if (actual == expected)  
        *addr = new;  
    return actual;  
}
```

```
void acquire(lock_t *lock) {  
    while(CompareAndSwap(&lock->flag, 0, 1) == 0) ;  
    // spin-wait (do nothing)  
}
```

Chat with neighbors for two mins....

Chat for a minute...

```
int xchg(int *addr, int newval) {  
    int old = *addr;  
    *addr = newval;  
    return old;  
}
```

```
int CompareAndSwap(int *addr,  
int expected, int new) {  
    int actual = *addr;  
    if (actual == expected)  
        *addr = new;  
    return actual;  
}
```

```
a = 1  
int b = xchg(&a, 2)  
int c = CAS(&b, 2, 3)  
int d = CAS(&b, 1, 3)
```

Final values?

LOCK IMPLEMENTATION GOALS

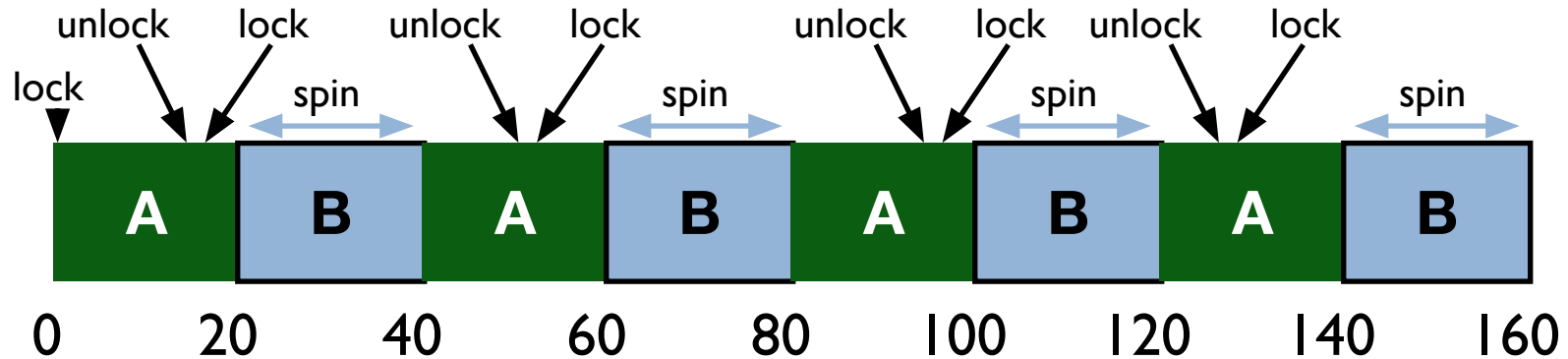
Correctness

- *Mutual exclusion*
Only one thread in critical section at a time
- *Progress* (deadlock-free)
If several simultaneous requests, must allow one to proceed

Fairness: does each thread have a fair shot at acquiring? Does anybody starve?

Performance: CPU is not used unnecessarily (spinning)

BASIC SPINLOCKS ARE UNFAIR



Scheduler is unaware of locks/unlocks!
B is unlucky - never is able to acquire lock

FAIRNESS: TICKET LOCKS

Idea: reserve each thread's turn to use a lock.

Each thread spins until their turn.

Use new atomic primitive, fetch-and-add

```
int FetchAndAdd(int *ptr) {  
    int old = *ptr;  
    *ptr = old + 1;  
    return old;  
}
```

Acquire: Grab ticket; Spin while not thread's ticket != turn

Release: Advance to next turn

TICKET LOCK IMPLEMENTATION

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
}
```

```
void lock_init(lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn = 0;  
}
```

```
void acquire(lock_t *lock) {  
    int myturn = FAA(&lock->ticket);  
    // spin  
    while (lock->turn != myturn);  
}
```

```
void release(lock_t *lock) {  
    lock->turn = lock->turn + 1  
}
```

Example

A.lock myturn = 0, ticket = 1

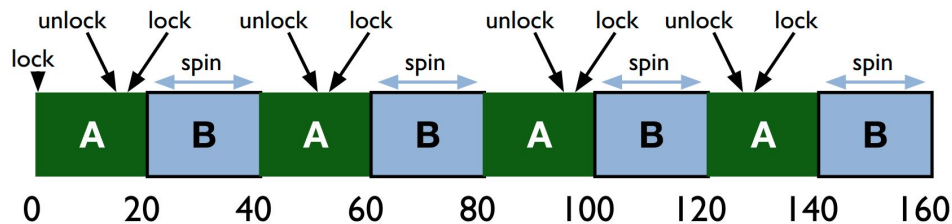
A.unlock turn =1

A. lock myturn = 1, ticket = 2

B.lock myturn =2, ticket = 3

A.unlock turn = 2

A.lock myturn=3, ticket = 4 // A cannot run now!

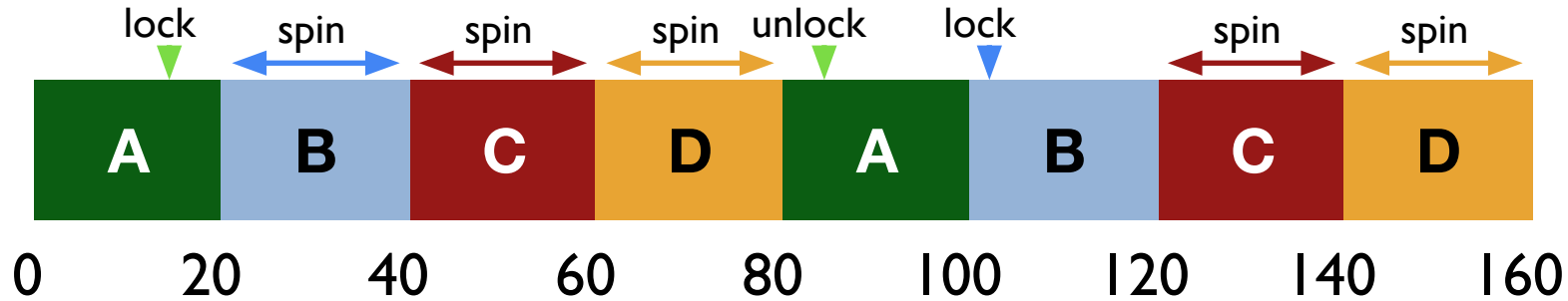


SPINLOCK PERFORMANCE

Slow when...

- one CPU
- locks held a long time
- disadvantage: spinning is wasteful

CPU SCHEDULER IS IGNORANT



CPU scheduler may run **B, C, D** instead of **A**
even though **B, C, D** are waiting for **A**

Solution: Yield (voluntarily deschedule yourself)

TICKET LOCK WITH YIELD

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
}
```

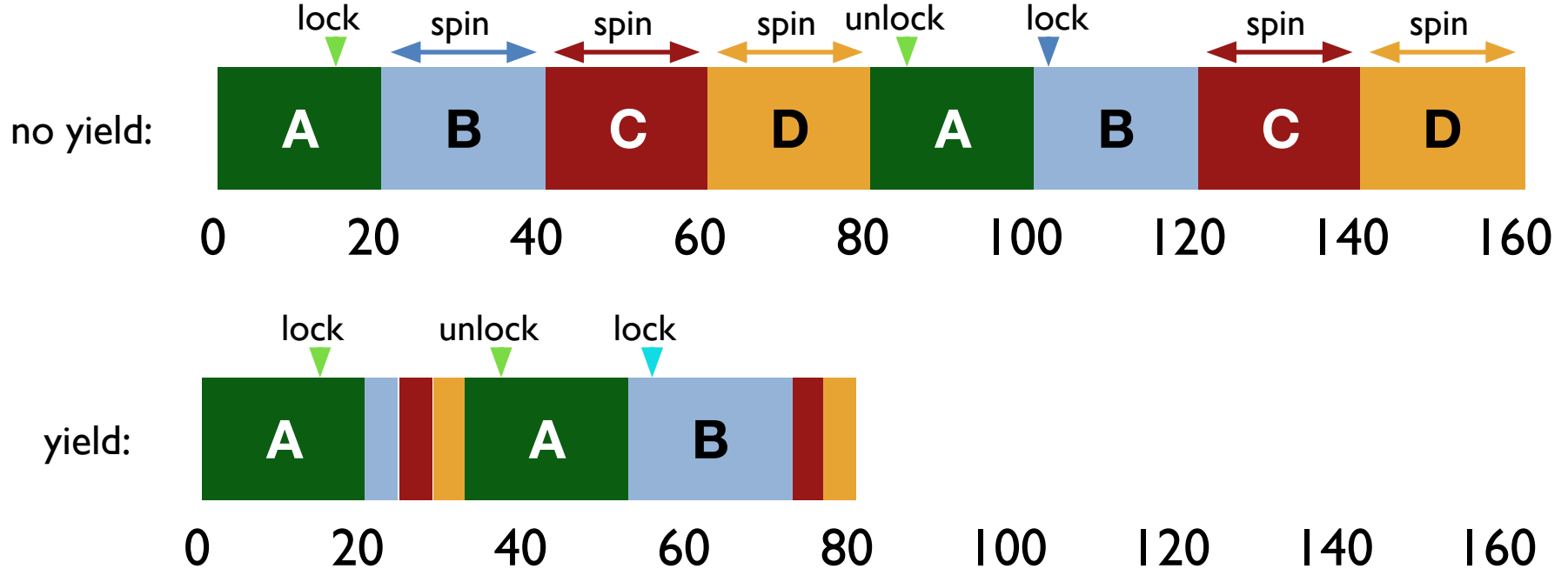
```
void lock_init(lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn = 0;  
}
```

```
void acquire(lock_t *lock) {  
    int myturn = FAA(&lock->ticket);  
    while (lock->turn != myturn)  
        yield();  
}
```

```
void release(lock_t *lock) {  
    lock->turn = lock->turn + 1  
}
```

`yield()` voluntarily relinquishes the CPU for remainder of time slice, but process remains READY

Yield Instead of Spin



YIELD VS SPIN

Waste of CPU cycles?

Without yield: $O(\text{threads} * \text{time_slice})$

With yield: $O(\text{threads} * \text{context_switch})$

While yield is better than spinning, with high contention, it can also be bad

Problem: the thread is still in READY state (and so can be scheduled on CPU)

Next improvement: Block and put thread on waiting queue

Lock Implementation: Block when Waiting

Remove waiting threads from scheduler runnable queue

Scheduler runs any thread that is **runnable**

Support in Solaris OS: `park()`, `unpark()`

Support in Linux: `futex` (fast userspace mutex)